IP 摄像头模块 —— 演讲稿

期末展示 · 第一部分 · 第 05–15 页 · 约 5½–6 分钟

ZHU Kecheng · SD5531 第 02 组 · 2026 年 11 月 6 日

一句话讲清逻辑

> 这一部分只回答一个问题——摄像头怎么决定录什么？一段有用的录像要满足两个要求：(1) 录到动作的开头；(2) 录的是守宫，而不是人或灯光。先讲装置，再分别讲怎么满足这两个要求，最后讲结果。

结构

页	部分	要点
05	**提出问题**	摄像头怎么决定录什么?
06–07	**装置**	PoE 摄像头 → ROCK 4C+ 录像；主码流录像，子码流分析
08	**要求一 · 录到开头**	一直在录：保留动作前 4 秒、动作后 5 秒
09–12	**要求二 · 只录守宫**	三个阶段：任何变化 → 检测区域 → 守宫本身
13–14	**怎样找到守宫**	区分蜥蜴和桌面靠的是颜色，不是亮度
15	**结果与下一步**	两个要求在实验室都达到了；下一步测真守宫

要记住的三个数字

4–6 秒
第一次动作前保留的画面

86 → 0
不该录的片段：第一阶段在学校 86 段 → 第三轮只有手的片段 0 段

0.23 秒
摄像头到 ROCK 的延迟

怎么用这份讲稿：每页标题下的绿色一行，是这一页要让听众记住的唯一要点。方括号 [ ] 里是提示，不用念出来。箭头（→）标出连接下一部分的过渡句。

讲稿

第 05 页 IP 摄像头模块

约 20 秒

> 要点 **一个问题：摄像头怎么决定录什么？**

我是 Kecheng，我来介绍 IP 摄像头模块。这一部分只回答一个问题：摄像头怎么决定录什么？

一段有用的录像要满足两个要求。第一，要录到动作的开头。第二，录的必须是守宫，而不是人或灯光。我先介绍装置，然后讲怎么满足这两个要求，最后讲结果和局限。

第 06 页 硬件连接与供电

约 30 秒

> 要点 **一根网线给摄像头供电；ROCK 4C+ 负责录像。**

这是我们真实的接线。摄像头通过一根网线从 PoE 供电器取得 48 伏电，同时传视频。供电器、ROCK 4C+ 和 MacBook 都接在一台千兆交换机上。ROCK 负责录像，Mac 保存录好的片段。

有一个薄弱点：[指向右侧 USB-C] ROCK 没有自己的电源适配器，是靠 Mac 的 USB-C 口供电的。ROCK 两次突然停机，我们怀疑和这条供电线路有关，但还没有证实。

第 07 页 两路码流，一个事件一段录像

约 35 秒

> 要点 **主码流用来录像，小的子码流用来分析。**

摄像头发两路码流，我们让它们各做一件事。2560×1440 的主码流只复制、不重新编码，所以录像几乎不占资源。小的子码流缩小到 160×90 做分析，判断事件什么时候开始、什么时候结束。然后剪出片段，用 SHA-256 核对后转存到 Mac。

[指向延迟测试] 我们让摄像头拍 Mac 屏幕上的时钟来测延迟：摄像头到 ROCK 是 0.23 秒。检测加跟踪只用了 ROCK 大约 13% 的 CPU，余量很充足。

→ 有了这套装置，先看第一个要求：录到动作的开头。

第 08 页 保留动作之前的几秒

约 40 秒

> 要点 **要求一：一直在录——保留动作前 4 秒、动作后 5 秒。**

如果检测到动作才开始录，开头一定会丢：检测最多要 0.65 秒，连接要 0.4 秒，FFmpeg 还要大约 2 秒才开始写文件。

所以我们的录像程序从不停止。它把画面切成 2 秒一段，写进内存里 60 秒的环形缓冲。事件结束时，把“第一次动作前 4 秒”到“最后一次动作后 5 秒”的分段拼起来，不重新编码。[指向片段] 这段实拍里，蜥蜴在第 5.1 秒被拉动，片段里还能看到它被拉动之前的样子。已经有 97 个片段这样保存并核对过。

→ 开头的问题解决了。第二个要求更难：只录守宫。

第 09 页 第一阶段 · 任何变化都算

约 25 秒

> 要点 **每个变化的像素都算，录得太多：2.5 小时 86 段。**

我们分三个阶段做到。第一阶段：两帧之间有变化的像素都算。速度快，但分不清是什么在动。在学校，两个半小时录了 86 段，画面里根本没有守宫——人、手、灯光和自动曝光都被当成了运动。

第 10 页 第二阶段 · 检测区域

约 30 秒

> 要点 **只看饲养箱范围，忽略整幅画面的变化。**

第二阶段告诉它该看哪里。我们在网页上画出饲养箱的范围，范围外的变化一律不算。如果整幅画面 30% 以上同时变化，就是光线或曝光变化，不算运动。另外，运动要持续半秒才算。

结果：区域外零次误触发，一次 31.4% 的整幅画面变化也被忽略了。但还剩一个问题：**[指向第三张图]** 手伸进区域里，仍然会录。

第 11 页 第三阶段 · 锁定守宫

约 30 秒

> 要点 **触发条件是守宫本身，而不是任何变化。**

所以第三阶段，触发条件变成守宫本身。只有被跟踪的守宫真的移动，才开始录：整个框在一秒内至少平移 2 个像素，而且两侧边要朝同一个方向移动。影子只会让一侧边变长，所以不算。

旧的区域规则还在运行，但只写进日志，这样可以在同一段视频上比较两种规则。

第 12 页 第三阶段 · 实验室测试

约 35 秒

> 要点 **实验室三轮测试：只有手的片段 3 → 2 → 0。**

我们在实验室分三轮测试第三阶段，每一次失败都变成了一条规则。第一轮：录到 3 段只有手的片段——像素闪动被当成了移动，所以我们要求真正的位移，并持续半秒。第二轮：2 段——手的影子和守宫连成一块，把框拉走了，所以我们加了“整个框平移”、重叠检查和大小限制。

第三轮：0 段。手在区域里停了 8 秒，旧规则触发了 87 帧，但一段都没录，三段守宫片段全部保留。

→ 这一切的前提是知道守宫在哪里。下面讲我们怎么找到它。

第 13 页 守宫跟踪 · 方法

约 30 秒

> 要点 **区分蜥蜴和桌面靠的是颜色，不是亮度。**

跟踪程序先用静止的画面学出背景，找出和背景不同的像素，把它们连成块，然后锁定那个在动的块。

关键是颜色。在亮度上，绿色蜥蜴和桌面几乎是同一个灰度——157 对 156 到 186——所以一开始框只框住了尾巴。在颜色通道上，两者差得很远。跟踪程序是纯 Python，每帧约 8 毫秒。

第 14 页 守宫跟踪 · 结果

约 20 秒

> 要点 **蜥蜴移动和静止时，框都能跟住它。**

加入颜色后，抽查 6 帧里有 5 帧框住了整只蜥蜴；蜥蜴静止 55 秒，框一直在它身上，没有误判为“丢失”。局限：蜥蜴被快速拿走时，框可能在原地停留大约 2 秒。

第 15 页 结果、局限与下一步

约 35 秒

> 要点 **两个要求在实验室都达到了；下一步是测真守宫。**

总结一下：两个要求在实验室都达到了。片段保留第一次动作前 4 到 6 秒；区域外的变化、光线变化和区域里的手，都不会再触发录像。

还没测的：真守宫——所有测试都用的是玩具蜥蜴；用摄像头红外模式的夜间录像；以及超过几个小时的长时间运行。下一步：在真实饲养箱里测试，做夜间测试，给 ROCK 配独立电源后跑 24 小时，并把片段和热成像事件关联起来。

→ 这就引出下一部分：热成像传感器。[交给下一位同学]

时间不够时（约 4 分钟）

- 第 07 页：只讲两路码流各做什么和 0.23 秒，跳过 CPU 数字。
- 第 09、10 页：各一句——“2.5 小时 86 段”和“区域外不算，但区域里的手仍会录”。
- 第 12 页：只讲“只有手的片段 3 → 2 → 0”。
- 第 13 页：只讲颜色这一点，跳过五个步骤。

可能被问到的问题

为什么视频不用 ESP32-C3?	它没有以太网，内存放不下一帧 2560×1440 的画面，也不能解码 H.264，所以录像放在 ROCK 4C+ 上。
真守宫能用吗?	还没测。所有测试都用玩具蜥蜴，进真实饲养箱测试是下一步的第一项。
第 08、10、12、14 页的画面是实时的吗?	不是。是回放：把录下的片段重新送进同一套检测和跟踪程序，逐帧画框。
31.4% 的变化是什么引起的?	录像里看不出原因；规则把它当作环境变化忽略了。

事实依据：project-camera/experiments（2026 年 10 月 30 日与 11 月 5 日）。数字与 Figma “Final PPT” 第 05–15 页一致。